

第3章 SQL语言（下）

3.1 SQL语言概述

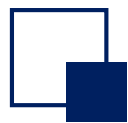
3.2 简单的SQL-DDL/DML: 创建数据库

3.3 SQL-DML之查询SELECT, FROM, WHERE, 多表查询

3.4 SELECT其他用法; 统计计算; GROUP BY+HAVING语句

3.5 子查询（嵌套查询）

3.6 集合操作



3.3 SQL-DML之查询SELECT

---- 结果计算与聚集函数

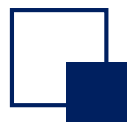
➤ 前面介绍的SELECT-FROM-WHERE语句中，SELECT子句后面不仅可以是列名，而且可以是一些计算表达式或聚集函数，表明在选择和投影的同时直接进行一些运算，如下所示：

```
SELECT 列名 | expr | agfunc(列名)  [[, 列名 | expr | agfunc(列名) ] ... ]
```

```
FROM 表名1 [, 表名2 ... ]
```

```
[ WHERE 检索条件 ] ;
```

➤ 计算表达式可以是常量、列名或由常量、列名、特殊函数及算术运算符构成的算术运算式。特殊函数的使用需结合各自DBMS的说明书。



3.3 SQL-DML之查询SELECT

---- 结果计算与聚集函数(续)

➤ 例如，有计算表达式投影列的示例：

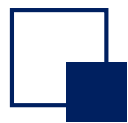
➤ 求有差额(差额>0)的任意两位教师的薪水差额

```
SELECT T1.tname AS TR1, T2.tname AS TR2, T1.salary - T2.salary  
FROM Teacher T1, Teacher T2  
WHERE T1.salary > T2.salary;
```

➤ 计算每位教师需要缴纳的税款。税率按20%计算。收入1000及以下的不纳入结果。

```
SELECT tname, salary*0.2 AS TAX  
FROM Teacher  
WHERE salary>1000;
```

Tname	TAX
赵三	240.00
赵四	280.00
赵六	220.00



3.3 SQL-DML之查询SELECT

---- 结果计算与聚集函数(续)

- SQL提供了五个作用在简单列值集合上的内置聚集函数agfunc, 分别是:
COUNT、SUM、AVG、MAX、MIN
- SQL聚集函数的参数类型、结果类型与作用如下:

Name	Argument type	Result type	Description
Count	any (can be *)	numeric	count of occurrences
sum	numeric	numeric	sum of arguments
avg	numeric	numeric	average of arguments
max	char or numeric	same as arg	maximum value
min	char or numeric	same as arg	minimum value

3.3 SQL-DML之查询SELECT

---- 结果计算与聚集函数(续)

- 求教师的工资总额

```
SELECT SUM(salary) FROM Teacher;
```

SUM(Salary)
4700.00

- 求计算机系教师的工资总额

```
SELECT SUM(salary) AS Total_Salary  
FROM Teacher T, Dept  
WHERE Dept.dname = '计算机' AND Dept.dno = T.dno;
```

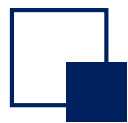
- 求数据库课程的平均成绩

```
SELECT AVG(score) FROM Course C, SC  
WHERE C.cname = '数据库' AND C.cno = SC.cno;
```

- 求教授课程的教师总数

```
SELECT COUNT(DISTINCT tno) FROM Course;
```

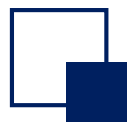
COUNT(*) 查询结果表的总行数
COUNT(X) 查询X列的行数（包括重复值）
COUNT(DISTINCT X) 查询X列不同值的个数



3.3 SQL-DML之查询SELECT

---- 结果计算与聚集函数(续)

- 下面的查询请求该如何表达呢?
- 求每一门课程的平均成绩
- 求每一个学生的平均成绩
- 以上问题，将在分组查询中解决



3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤

- 为解决同时求解若干个集合的聚集运算问题，引出了分组的概念。SQL可以将检索到的元组按照某一条件进行分类，具有相同条件值的元组划到一个组或一个集合中，这一过程就是分组过程。
- 分组可以在基本SELECT语句基础上引入分组子句来完成：

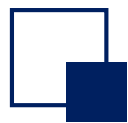
SELECT 列名 | **expr** | **agfunc**(列名) [[, 列名 | **expr** | **agfunc**(列名)] ...]

FROM 表名1 [, 表名2 ...]

[**WHERE** 检索条件]

[**GROUP BY** 分组条件] ;

- 分组条件可以是 列名1,
列名2, ...



3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

- 例如前面例子：求每一个学生的平均成绩

```
SELECT sno ,AVG(score) FROM SC  
GROUP BY sno ;
```

- 上述例子是按学号进行分组，即学号相同的元组划到一个组中并求平均值。
- 改变分组条件便形成了另外的结果：求每一门课程的平均成绩

```
SELECT cno ,AVG(score) FROM SC  
GROUP BY cno ;
```

注意！ 有GROUP BY语句存在时，SELECT中可以选择的列受到限制！

只能选取 (1) GROUP BY语句中包括的列，(2) 聚集函数。

Why?

3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

➤ 不同分组条件分组示例。分组之后，每组将被压缩为结果的一行

sno	Avg(score)
210110101	88.3
210110202	75
210110303	56
210110102	62.3

GROUP BY

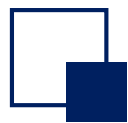
sno

sno	cno	score
210110101	001	92
210110101	002	85
210110101	003	88
210110202	002	90
210110202	003	80
210110202	001	55
210110303	003	56
210110102	001	54
210110102	002	85
210110102	003	48

GROUP BY

cno

sno	cno	score
210110101	001	92
210110202	001	55
210110102	001	54
210110101	002	85
210110202	002	90
210110102	002	85
210110101	003	80
210110101	003	88
210110203	003	56
210110102	003	48



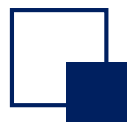
3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

- 求不及格课程超过两门的同学的学号

```
SELECT sno FROM SC  
WHERE score < 60 AND COUNT(*)>2 GROUP BY sno ;
```

聚集函数是不允许用于WHERE子句中的：WHERE子句是对每一元组进行条件过滤，而不是对集合进行条件过滤



3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

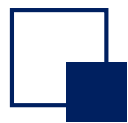
- 若要对集合(即分组)进行条件过滤, 可使用HAVING子句
- HAVING子句, 又称为分组过滤子句。需要有GROUP BY子句支持, 换句话说, 没有GROUP BY子句, 便不能有HAVING子句。

SELECT 列名 | **expr** | **agfunc**(列名) [[, 列名 | **expr** | **agfunc**(列名)] ...]

FROM 表名1 [, 表名2 ...]

[**WHERE** 检索条件]

[**GROUP BY** 分组条件 [**HAVING** 分组过滤条件]] ;



3.3 SQL-DML之查询SELECT

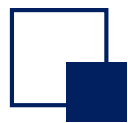
---- 分组查询与分组过滤(续)

- 例如求不及格课程超过两门的同学的学号

```
SELECT sno FROM SC WHERE score < 60  
GROUP BY sno HAVING COUNT(*) > 2;
```

- 再如求有10人以上不及格的课程号

```
SELECT cno FROM SC WHERE score < 60  
GROUP BY cno HAVING COUNT(*) > 10;
```



3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

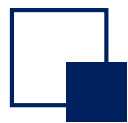
➤ **HAVING**子句与**WHERE**子句表达条件的区别如下图示例

每一分组检查满足与否的条件要用**HAVING**子句表达

注意:不是每一行都检查,
所以使用**HAVING**子句一定要有**GROUP BY**子句

sno	cno	score
210110101	001	92
210110101	002	85
210110101	003	88
210110202	002	90
210110202	003	80
210110202	001	55
210110303	003	56
210110102	001	54
210110102	002	85
210110102	003	48

每一行都要检查满足与否的条件要用 **WHERE** 子句表达



3.3 SQL-DML之查询SELECT

---- 经典问题:

- 找到年龄最大的同学，给出其学号、姓名和年龄。如何查询？

SELECT sno, sname, max(sage) FROM student;

SELECT sno, sname, sage FROM student WHERE sage = max(sage);

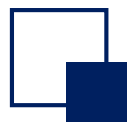
以上查询均不正确

- 类似问题:

- 找到‘001’号课程分数最低的同学，给出其学号、姓名以及‘001’课的分。

- 找到平均分最低的同学，给出其学号、姓名、平均分。

- 注意：每个查询都可能有多条符合条件的结果。查询需要保证结果完整性



3.3 SQL-DML之查询SELECT

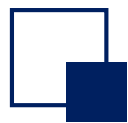
---- 分组查询与分组过滤(续)

- 分组查询仍需要注意语义问题
- 例如求有两门以上不及格课程的同学的学号及其平均成绩

```
SELECT sno ,Avg(score) FROM SC WHERE score < 60
```

```
GROUP BY sno HAVING COUNT(*)>2;
```

- 上述写法正确吗?

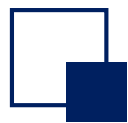


3.3 SQL-DML之查询SELECT

---- 分组查询与分组过滤(续)

➤ 前述写法是不正确的，它求出的是那两门不及格课程的平均成绩，而不是该同学所有课程的平均成绩，后者是题目要求的。因此正确写法为：

```
SELECT sno ,AVG(score) FROM SC
WHERE sno IN
      ( SELECT  sno  FROM SC
        WHERE score < 60
        GROUP BY sno HAVING COUNT(*)>2 )
GROUP BY sno ;
```

3.3 SQL-DML之查询SELECT

---- 子查询(嵌套查询)

- 在现实中，很多情况下，六个语句的组合无法完成查询任务。有时需要通过多个步骤完成查询，每个步骤都需要一个SELECT语句查询获得中间结果。
- 使用子查询/嵌套查询来实现。
- 出现在其他子句中的SELECT...语句被称为子查询(subquery)，子查询返回了一个集合，可以通过与这个集合的比较来确定另一个查询的结果。
- 子查询通常可以出现在FROM, WHERE和HAVING子句里，用以下关键词与对应的语句连接起来
 - IN/NOT IN;
 - SOME/ALL
 - EXISTS/NOT EXISTS

3.3 SQL-DML之查询SELECT

---- 子查询之类型1：IN谓词

- 例如：求选修了001号课程的学生们的学号和姓名
 - 步骤1：从SC表里找到所有选001课程的学号集合；
 - 步骤2：从Student表里查出学号在该集合里的学生信息

```
SELECT sno, sname FROM Student
WHERE sno IN (SELECT sno FROM SC WHERE cno = '001');
```

- 再例如：求既学过001号课程, 又学过002号课程的学生们的学号

```
SELECT      sno      FROM SC
WHERE cno = '001' AND
      sno IN (SELECT sno FROM SC WHERE cno = '002');
```

用表连接可否?

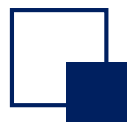
步骤1 (子查询)

SNo
98030101
98040202

表 (Table) : SC		
SNo	CNo	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56

步骤2 (主查询)

Student					
SNo	Sname				
98030101	张三				
98030102	张四				
98030103	张五				
98040201	王三				
98040202	王四				
98040203	王五				



3.3 SQL-DML之查询SELECT

---- 子查询之类型1: IN谓词

- 第1种类型的子查询: IN子查询。其基本语法为:

表达式 **[NOT]IN** (子查询)

- 语义: 判断某一表达式的值是否在子查询的结果中。语法中, 表达式的最简单形式就是列名或常数。
- 例如: 列出张三、王三同学的所有信息

```
SELECT * FROM Student
```

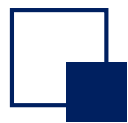
```
WHERE sname IN ( '张三', '王三' );
```

//此处直接使用了某一子查询的结果集合。如果该集合是已知的固定的, 可以如上直接书写

- 上述示例相当于

```
SELECT * FROM Student
```

```
WHERE sname = '张三' OR sname = '王三';
```



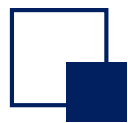
3.3 SQL-DML之查询SELECT

---- 子查询之类型1: IN谓词(续)

- 再例如：列出没学过李明老师讲授课程的所有同学的姓名？

```
SELECT sname FROM Student
WHERE sno NOT IN ( SELECT sno FROM SC, Course C, Teacher T
                    WHERE T.tname = '李明'
                      AND SC.cno = C.cno
                      AND T.tno = C.tno );
```

- 练习一下：列出所有没教过‘赵五’的老师姓名？



3.3 SQL-DML之查询SELECT

---- 非相关子查询

- 如下所示，将SELECT语句区分为内层和外层

SELECT sname

FROM Student

WHERE sno NOT

IN (SELECT

sno **FROM** SC, Course C, Teacher T

WHERE T.tname = '李明' AND

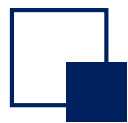
SC.cno = C.cno AND T.tno = C.tno);

外层查询

内层查询

内层查询独立进行，没有涉及任何外层查询相关信息的子查询被称为非相关子查询。

- 前面的子查询示例都是非相关子查询



3.3 SQL-DML之查询SELECT

---- 相关子查询

- 有时，内层查询需要依靠外层查询的某些参量作为限定条件才能进行，这样的子查询称为相关子查询。
- 外层向内层传递的参量需要使用外层的表名或表别名来限定
- 例如：求学过001号课程的同学的姓名

```
SELECT sname
FROM Student Stud
WHERE sno IN ( SELECT sno
                FROM SC
                WHERE sno = Stud.sno AND cno = '001' );
```

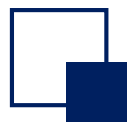
➤ 注意：相关子查询只能由外层向内层传递参数，而不能反之；这也称为变量的作用域原则。

主查询

Student (Stud)					
SNo	Sname				
98030101	张三				
98030102	张四				
98030103	张五				
98040201	王三				
98040202	王四				
98040203	王五				

表 (Table) : SC		
SNo	CNo	Score
98030101	001	92
98030101	002	85
98030101	003	88
98040202	002	90
98040202	003	80
98040202	001	55
98040203	003	56

sno = Stud.sno

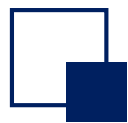


3.3 SQL-DML之查询SELECT

➤ 求有两门以上不及格课程的同学的学号及其平均成绩

```
SELECT sno ,Avg(score) FROM SC WHERE score < 60  
GROUP BY sno HAVING COUNT(*)>2;
```

上述写法正确吗?



3.3 SQL-DML之查询SELECT

➤ 前述写法是不正确的，它求出的是那两门不及格课程的平均成绩，而不是该同学所有课程的平均成绩，后者是题目要求的。因此正确写法为：

```
SELECT sno ,AVG(score) FROM SC
WHERE sno IN
      ( SELECT  sno  FROM SC
        WHERE score < 60
        GROUP BY sno HAVING COUNT(*)>2 )
GROUP BY sno ;
```




互动小问题

- 求既学过001号课程, 又学过002号课程的学生学号, 能否用相关子查询来表达?



互动小问题

➤ 相关子查询和非相关子查询的区别是什么？

3.3 SQL-DML之查询SELECT

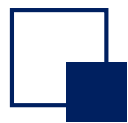
---- 子查询之类型2: θ SOME / θ ALL谓词

- 第2种类型的子查询: θ SOME / θ ALL子查询。其基本语法为:

表达式 θ SOME(子查询)

表达式 θ ALL (子查询)

- 语法中, θ 是比较运算符: $<$, $>$, $>=$, $<=$, $=$, $<>$ 。
- 语义: 将表达式的值与子查询的结果进行比较:
 - 如果表达式的值与子查询结果中至少某一个值相比较满足 θ 关系, 则“表达式 θ SOME(子查询)”的结果便为真;
 - 如果表达式的值与子查询结果的所有值相比较都满足 θ 关系, 则“表达式 θ ALL(子查询)”的结果便为真;



3.3 SQL-DML之查询SELECT

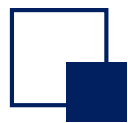
---- 子查询之类型2: θ SOME / θ ALL谓词(续)

- (之前的经典问题) 找出年龄最大的学生的学号、姓名、年龄

```
SELECT Sno, Sname, Sage FROM Student  
WHERE Sage >= ALL ( SELECT Sage FROM Student );
```

- 再例如: 找出工资最低的教师姓名

```
SELECT tname FROM Teacher  
WHERE salary <= ALL ( SELECT salary FROM Teacher );
```



3.3 SQL-DML之查询SELECT

---- 子查询之类型2: θ SOME / θ ALL谓词(续)

➤ 再例如: 找出曾经有不及格成绩的学生姓名

```
SELECT sname FROM Student
WHERE 60 > SOME ( SELECT score FROM SC
                  WHERE sno = Student.sno ); (相关子查询)
```

➤ 再例如: 找出所有课程都不及格的学生姓名

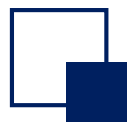
```
SELECT sname FROM Student
WHERE 60 > ALL ( SELECT score FROM SC
                 WHERE sno = Student.sno ); (相关子查询)
```



练习

- 请同学们书写满足下述条件的查询语句
 - 找出001号课成绩最高的所有学生的学号

```
Select SNo
From SC
Where Cno ='001' and Score >= all (
    Select Score
    From SC
    Where CNo='001' );
```



3.3 SQL-DML之查询SELECT

---- 子查询之类型2: θ SOME / θ ALL谓词(续)



如下两种表达方式含义是相同的:

表达式= SOME(子查询) 与 表达式IN (子查询)

➤ 例如:

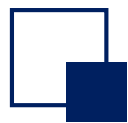
```
SELECT sname FROM Student Stud
WHERE sno IN ( SELECT sno FROM SC
WHERE sno = Stud.sno AND cno = '001' );
```

```
SELECT sname FROM Student Stud
WHERE sno = SOME ( SELECT sno FROM SC
WHERE sno = Stud.sno AND cno = '001' );
```



互动小问题

- 求既学过001号课程, 又学过002号课程的学生们的学号, 能否用 θ SOME 表达?



3.3 SQL-DML之查询SELECT

---- 子查询之类型2: θ SOME / θ ALL谓词(续)

➤ 然而下面两种表达方式含义却是不同的, 请注意

表达式 **NOT IN** (子查询) 与 表达式 $\neq$ **SOME**(子查询)

与**NOT IN**等价的是

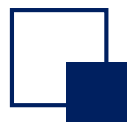
表达式 $\neq$ **ALL** (子查询)

互动小问题

➤ 检索选修了李明老师主讲课程的所有同学的姓名

```
SELECT DISTINCT sname FROM Student, SC, Course, Teacher  
WHERE SC.cno = Course.cno AND SC.sno = Student.sno  
AND Course.tno = Teacher.tno AND tname = '李明';
```

除了以上表连接？还能怎么表达？



3.3 SQL-DML之查询SELECT

---- 子查询之类型3: EXISTS谓词

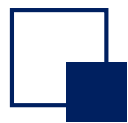
- 第3种类型的子查询: **EXISTS**子查询。其基本语法为:

[NOT] EXISTS (子查询)

- 语义: 子查询结果中有无元组存在。
- 例如: 检索选修了李明老师主讲课程的所有同学的姓名

```
SELECT DISTINCT sname FROM Student
WHERE EXISTS ( SELECT * FROM SC, Course, Teacher
                WHERE SC.cno = Course.cno AND SC.sno = Student.sno AND
                      Course.tno = Teacher.tno AND tname = '李明' );
```

- 互动小问题: 上例为什么用DISTINCT?



3.3 SQL-DML之查询SELECT

---- 子查询之类型3: EXISTS谓词(续)

再例如: 列出没学过李明老师讲授课程的所有同学的姓名?

```
SELECT sname FROM Student
```

```
WHERE NOT EXISTS
```

//不存在

```
( SELECT * FROM Course, SC, Teacher
```

//学过一门课程

```
WHERE tname='李明' AND Course.tno =Teacher.tno AND
```

```
Course.cno = SC.cno AND Sno = Student.sno );
```



互动小问题

➤ 上例还能怎么表达？

互动小问题

➤ 求既学过001号课程, 又学过002号课程的学生们的学号, 请用 EXISTS 语句描述。

```
Select SNo From SC
Where Cno ='001' and exists
(Select *
  From SC SC1
   Where Cno ='002' and SNo=SC.SNo) ;
```

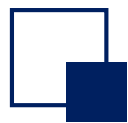
互动小问题

---- 子查询之类型3: EXISTS谓词(续)

- 然而NOT EXISTS却可以实现很多新功能
- 例如：检索学过001号教师主讲的所有课程的所有同学的姓名

```
SELECT sname FROM Student, SC, Course  
WHERE SC.cno = Student.sno AND SC.cno = Course.cno AND tno = '001';
```

是否可以用以上语句来表达？



3.3 SQL-DML之查询SELECT

---- 子查询之类型3: EXISTS谓词(续)

- 然而NOT EXISTS却可以实现很多新功能
- 例如：检索学过001号教师主讲的所有课程的所有同学的姓名

```
SELECT sname FROM Student  
WHERE NOT EXISTS
```

//不存在

```
( SELECT * FROM Course
```

//有一门001教师主讲课程

```
WHERE Course.tno = '001' AND NOT EXISTS
```

//该同学没学过

```
( SELECT * FROM SC
```

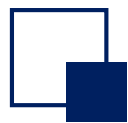
```
WHERE sno = Student.sno AND cno = Course.cno ) );
```

- 上述语句的意思是：不存在有一门001号教师主讲的课程该同学没学过



互动小问题

➤ 上例用关系代数除怎么操作？



3.3 SQL-DML之查询SELECT

---- 子查询之类型3: EXISTS谓词(续)

➤ 再例如：列出至少学过21030101号同学学过**所有**课程的同学的学号

```
SELECT DISTINCT sno FROM SC SC1
```

```
WHERE NOT EXISTS
```

//不存在

```
( SELECT * FROM SC SC2
```

//有一门课程

```
WHERE SC2.sno = '21030101' AND NOT EXISTS
```

//该同学没学过

```
( SELECT * FROM SC
```

```
WHERE cno = SC2.cno AND sno = SC1.sno ));
```



互动小问题

➤ 上例用关系代数除怎么操作？

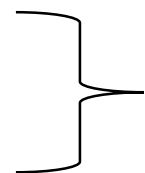
复习、小结

- 子查询与主查询连接的方式:

IN/NOT IN,

θ SOME/ALL,

EXISTS/NOT EXISTS



(可为相关或非相关子查询)

(正常情况下均为相关子查询)

- 子查询与主查询独立性: 非相关子查询, 相关子查询
- 以上均为在**WHERE**子句中的子查询

互动小问题

- 查询学过李明老师主讲的所有课程的同学姓名, 请用SQL语句表达。

“所有”：用关系代数 $\div$ 比较好做！

SQL中则可以用双重NOT EXISTS子查询嵌套

Select Sname From Student S

Where not exists

(Select * From Teacher T, Course C

Where T.Tno=C.Tno and T.Tname='李明' and not exists

(Select * From SC

where Sno=S.Sno and Cno=C.Cno));

//对某位同学, 不存在这样一门课:

// 这门课是李明讲过的且不存在:

//该同学选该门课的记录

于是, 该同学一定选过李明老师的所有课

课后小练习

---- 子查询之类型3: EXISTS谓词(续)

➤ 再例如:

➤ 已知: SPJ(sno, pno, jno, qty), 其中sno供应商号, pno零件号, jno工程号, Qty数量, 列出至少用了供应商S1供应的全部零件的工程号。

```
SELECT DISTINCT jno FROM SPJ SPJ1 WHERE  
NOT EXISTS  
( SELECT * FROM SPJ SPJ2  
WHERE SPJ2.sno = 'S1' AND NOT EXISTS (  
SELECT * FROM SPJ SPJ3  
WHERE SPJ3.pno = SPJ2.pno  
AND SPJ3.jno = SPJ1.jno ) );
```

//不存在

//有一种S1的零件

//该工程没用过

---- 其他子句的子查询

➤ FROM: 将子查询结果作为主查询的输入表，进行笛卡尔积或连接操作

SELECT S1.Sno, S1.Sname, S1.Sage

FROM Student S1, (SELECT MAX(Sage) as MaxAge FROM Student) S2

WHERE S1.Sage = S2.MaxAge;

Student					
SNo	Sname	Gender	Sage	DNo	Sclass
98030101	张三	男	20	03	980301
98030102	张四	女	20	03	980301
98030103	张五	男	19	03	980301
98040201	王三	男	20	04	980402
98040202	王四	男	21	04	980402
98040203	王五	女	19	04	980402

MaxAge
21

S1						S2
SNo	Sname	Gender	Sage	DNo	Sclass	MaxAge
98040202	王四	男	21	04	980402	21

S1		
SNo	Sname	Sage
98040202	王四	21

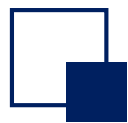
找到年龄最大学生的学号，姓名以及年龄



---- 其他子句的子查询

- HAVING: 多为 θ SOME/ALL子查询
- 查询平均分最高的学生的学号、姓名和平均分

```
SELECT S1.Sno, S1.Sname, S1.Sage  
FROM SC, Student S  
WHERE SC.Sno = S.Sno  
GROUP BY SC.Sno, S1.Sname, S1.Sage  
HAVING AVG(Score) >= ALL ( SELECT AVG(Score) FROM SC SC1  
                           GROUP BY SC1.Sno);
```

3.3 SQL-DML之查询SELECT

---- 并、交、差的处理

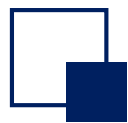
- 关系代数中有并运算、交运算和差运算，SQL语言也能实现其运算
- SQL语言中为并运算提供了运算符UNION，为交运算提供了运算符INTERSECT，为差运算提供了运算符EXCEPT，其基本语法形式为：

子查询{ UNION [ALL] | INTERSECT [ALL] | EXCEPT [ALL] 子查询 }

- 上述操作，在通常情况下是自动删除重复元组。但有时为了保留重复的元组，则需使用ALL保留字，ALL保留字是允许重复元组出现，具体使用如下：

□ 假设子查询1的一个元组出现m次，子查询2的一个元组出现n次，则 该元组在：

- ✓ 子查询1 UNION ALL 子查询2，出现 $m+n$ 次
- ✓ 子查询1 INTERSECT ALL 子查询2，出现 $\min(m,n)$ 次
- ✓ 子查询1 EXCEPT ALL 子查询2，出现 $\max(0, m-n)$ 次



3.3 SQL-DML之查询SELECT

---- 并、交、差的处理(续)

➤ SQL并运算示例:

➤ 求学过002号课的同学或学过003号课的同学学号

```
SELECT sno FROM SC WHERE cno = '002'
```

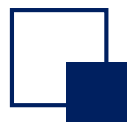
```
UNION
```

```
SELECT sno FROM SC WHERE cno = '003';
```

➤ 上述语句也可采用如下不用UNION的方式来进行

```
SELECT Sno FROM SC WHERE cno = '002' OR cno = '003';
```

```
SELECT Sno FROM SC WHERE cno IN ('002', '003');
```



3.3 SQL-DML之查询SELECT

---- 并、交、差的处理(续)

➤ 有时只能用UNION的方式

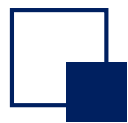
➤ 例如：两个表Customers(cid, cname, city, discnt), Agents(aid, aname, city, percent),

求：客户所在的或者代理商所在的城市

```
SELECT city FROM Customers
```

```
UNION
```

```
SELECT city FROM Agents;
```



3.3 SQL-DML之查询SELECT

---- 并、交、差的处理(续)

➤ SQL交运算示例:

➤ 求既学过002号课, 又学过003号课的同学学号

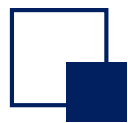
➤ **SELECT sno FROM SC WHERE cno = '002'**
INTERSECT

SELECT sno FROM SC WHERE cno = '003';

➤ 上述语句也可采用如下不用INTERSECT的方式来进行

SELECT sno FROM SC WHERE cno = '002' AND sno IN
(SELECT sno FROM SC WHERE cno = '003');

➤ 交运算符INTERSECT并没有增强SQL的表达能力, 没有INTERSECT, SQL也可以用其他方式表达同样的查询需求, 如上例所述。只是有了INTERSECT更容易表达一些, 但增加了SQL语言的不唯一性。



3.3 SQL-DML之查询SELECT

---- 并、交、差的处理(续)

- SQL差运算示例： 假定所有学生都有选课，求没学过002号课程的学生学号
- 上例可写成如下形式：所有学生 - 学过002号课的学生

```
SELECT DISTINCT sno FROM SC
```

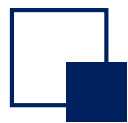
```
EXCEPT
```

```
SELECT sno FROM SC WHERE cno = '002';
```

差集关键字： **EXCEPT** (SQL Server, Postgres, MySQL), **MINUS** (Oracle DB), 二者等价

Please note that MySQL supported the `EXCEPT` operator starting from version 8.0.31. The `EXCEPT` operator is equivalent to the `MINUS` operator.

If you are using a lower version, you can refer to this tutorial to emulate the `MINUS` operator.



3.3 SQL-DML之查询SELECT

---- 并、交、差的处理(续)

- 前述语句也可采用如下不用EXCEPT的方式来进行

```
SELECT DISTINCT sno FROM SC SC1
```

```
WHERE NOT EXISTS
```

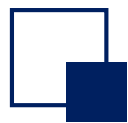
```
( SELECT * FROM SC WHERE cno = '002'
```

```
AND sno = SC1.sno );
```

- 差运算符EXCEPT也没有增强SQL的表达能力，没有EXCEPT，SQL也可以用其他方式表达同样的查询需求，如上例所述。只是有了EXCEPT更容易表达一些，但增加了SQL语言的不唯一性。



➤ 找到年龄最大的学生, 请用**EXCEPT**语句描述, 不使用子查询。



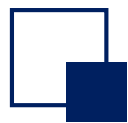
3.3 SQL-DML之查询SELECT

---- 空值处理

- 空值是其值不知道、不确定、不存在的值
- 数据库中有了空值，会影响许多方面，如影响聚集函数运算的正确性，不能参与算术、比较或逻辑运算等
- 例如：右下图所示表SC, 如果有某一记录为空值，则求001号课程的平均成绩？会是多少呢？
- 以前，很多DBMS将空值按默认值处理，如字符串类型则以空格来表示，而如数值类型则以0来表示，这也将会引起统计、计算上的不正确性。

SC

sno	cno	score
21030101	001	92
21040202	001	55
21030102	001	NULL



3.3 SQL-DML之查询SELECT

---- 空值处理(续)

➤ 在SQL标准中和许多现流行的DBMS中，空值被用一种特殊的符号NULL来标记，使用特殊的空值检测函数来获得某列的值是否为空值。

➤ 空值检测

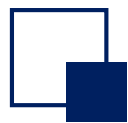
IS [NOT] NULL

测试指定列的值是否为空值

➤ 例如：找出年龄值为空的学生姓名

SELECT sname FROM Student WHERE sage IS NULL;

注意：上例条件不能写为 WHERE sage = NULL; 空值是不能进行运算的

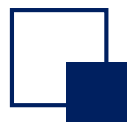


3.3 SQL-DML之查询SELECT

---- 空值处理(续)

➤ 现行DBMS的空值处理小结

- ❑ 除IS [NOT] NULL之外，空值不满足任何查找条件
- ❑ 如果NULL参与算术运算，则该算术表达式的值为NULL
- ❑ 如果NULL参与比较运算，则结果可视为FALSE。在SQL-92中可看成UNKNOWN
- ❑ 如果NULL参与聚集运算，则除COUNT(*)之外其它聚集函数都忽略NULL



3.3 SQL-DML之查询SELECT

---- 空值处理(续)

➤ 例如:

SELECT AVG(score) FROM SC;

上例的值(参考右图)为 $79.67 = (92 + 55 + 92)/3$ 。

➤ 再例如: 下面三个查询结果是否相同?

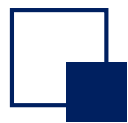
SELECT COUNT(*) FROM SC;

SELECT COUNT(score) FROM SC;

SELECT COUNT(DISTINCT score) FROM SC;

SC

sno	cno	score
21030101	001	92
21040202	001	55
21030102	001	NULL
21030102	002	92



3.3 SQL-DML之查询SELECT

---- 内连接、外连接

- 在SQL的高级语法中引入了内连接与外连接运算，具体形式如下：

SELECT 列名 [[, 列名] ...]

FROM 表名1 [NATURAL]

[INNER | { LEFT | RIGHT | FULL } [OUTER]] JOIN 表名2

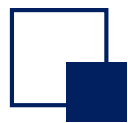
{ ON 连接条件 | USING (Colname {, Colname ...}) }

[**WHERE** 检索条件] ... ;

- 上例的连接运算由两部分构成：连接类型和连接条件

连接类型(四者选一)
INNER JOIN
LEFT OUTER JOIN
RIGHT OUTER JOIN
FULL OUTER JOIN

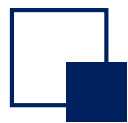
连接条件(三者选一)
NATURAL
ON <连接条件>
USING (Col ₁ , Col ₂ , ..., Col _n)



3.3 SQL-DML之查询SELECT

---- 内连接、外连接(续)

- INNER JOIN: 即关系代数中的 θ -连接运算 (内连接, INNER可略去)
- LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN: 即关系代数中的外连接运算
 - 如“表1 LEFT OUTER JOIN 表2”, 则连接后, 表1的任何元组t都会出现在结果表中, 如表2中有满足连接条件的元组s, 则t与s连接; 否则t与空值元组连接;
 - 如“表1 RIGHT OUTER JOIN 表2”, 则连接后, 表2的任何元组s都会出现在结果表中, 如表1中有满足连接条件的元组t, 则t与s连接; 否则s与空值元组连接;
 - 如“表1 FULL OUTER JOIN 表2”, 是前两者的并。



3.3 SQL-DML之查询SELECT

---- 内连接、外连接(续)

➤ 连接中使用 ON <连接条件> (θ -连接)

□ 出现在结果关系中的两个连接关系的元组取值满足连接条件，且公共属性出现两次

➤ 连接中使用 NATURAL (自然连接)

□ 出现在结果关系中的两个连接关系的元组在公共属性上取值相等，且公共属性只出现一次

➤ 连接中使用 USING ($Col_1, Col_2, \dots, Col_n$) (自然连接的特例)

□ ($Col_1, Col_2, \dots, Col_n$) 是两个连接关系的公共属性的子集，元组在($Col_1, Col_2, \dots, Col_n$)上取值相等，且($Col_1, Col_2, \dots, Col_n$)只出现一次

3.3 SQL-DML之查询SELECT

---- 内连接、外连接(续)

➤ INNER JOIN 示例: 求所有教师的任课情况并按教师号排序(没有任课的教师也需列在表中)

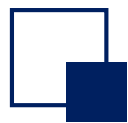
```
SELECT Teacher.tno, tname, cname FROM
Teacher INNER JOIN Course
ON Teacher.tno= Course.tno
ORDER BY Teacher.tno ASC;
```

任课情况（内连接）		
tno	tname	cname
001	赵三	数据库
001	赵三	编译原理
003	赵五	数据结构
003	赵五	C语言
004	赵六	高等数学

Course				
cno	cname	chours	credit	tno
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C语言	30	4.5	003
002	高等数学	80	12	004

Teacher			
tno	tname	dno	salary
001	赵三	01	1200
002	赵四	03	1400
003	赵五	03	1000
004	赵六	04	1100





3.3 SQL-DML之查询SELECT

---- 内连接、外连接(续)

➤ OUTER JOIN示例: 求所有教师的任课情况(没有任课的教师也需列在表中)

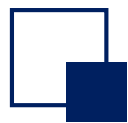
```
SELECT Teacher.tno, tname, cname
FROM Teacher LEFT OUTER JOIN Course
ON Teacher.tno = Course.tno
ORDER BY Teacher.tno ASC;
```

任课情况 (左外连接)		
tno	tname	cname
001	赵三	数据库
001	赵三	编译原理
002	赵四	null
003	赵五	数据结构
003	赵五	C语言
004	赵六	高等数学



Course				
cno	cname	chours	credit	tno
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C语言	30	4.5	003
002	高等数学	80	12	004

Teacher			
tno	tname	dno	salary
001	赵三	01	1200
002	赵四	03	1400
003	赵五	03	1000
004	赵六	04	1100



3.3 SQL-DML之查询SELECT

- OUTER JOIN、NOT EXISTS、EXCEPT有时可实现相同功能
- 找到没上过任何课的老师姓名

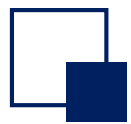
```
SELECT Teacher.tno , tname
FROM Teacher LEFT OUTER JOIN Course
      ON Teacher.tno = Course.tno
WHERE cname IS NULL;
```

```
SELECT tno , tname
FROM Teacher
EXCEPT

SELECT Teacher.tno , tname
FROM Teacher JOIN Course ON Teacher.tno = Course.tno;
```

```
SELECT tno , tname
FROM Teacher
WHERE NOT EXISTS (
      SELECT *
      FROM Course C
      WHERE Teacher.tno = C.tno );
```

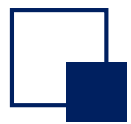
任课情况（左外连接）		
tno	tname	cname
001	赵三	数据库
001	赵三	编译原理
002	赵四	null
003	赵五	数据结构
003	赵五	C语言
004	赵六	高等数学



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之更新

- 元组新增INSERT: 新增一个或一些元组到数据库的Table中
 - 元组更新UPDATE: 对某些元组中的某些属性值进行重新设定
 - 元组删除DELETE: 删除某些元组
-
- SQL-DML既能单一记录操作, 也能对记录集合进行批更新操作
 - SQL-DML之更新操作需要利用前面介绍的子查询(Subquery)的概念, 以便处理“一些”、“某些”等。



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组新增INSERT

➤ 元组新增INSERT命令有两种形式

➤ 单一元组新增命令形式：插入一条指定元组值的元组

INSERT INTO 表名 [(列名[, 列名]...)]
VALUES (值[, 值]...);

➤ 批数据新增命令形式：插入子查询结果中的若干条元组。待插入的元组由子查询给出。

INSERT INTO 表名 [(列名[, 列名]...)]
子查询;



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组新增INSERT(续)

➤ 单一元组新增INSERT示例

```
INSERT INTO Teacher (tno , tname, dno , salary)  
VALUES ('005', '阮小七' , '03', '1250');
```

```
INSERT INTO Teacher  
VALUES ('006', '李小虎' , '03', '950');
```



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组新增INSERT(续)

➤ 批元组新增INSERT示例

➤ 如果我们新建立了一个Table: **St(sno , sname, avgScore)**, 然后将检索到的同学的平均成绩新增到该表中

```
INSERT INTO St (sno , sname, avgScore)
```

```
SELECT sno , sname, Avg(score) FROM Student, SC WHERE
```

```
Student.sno = SC.sno
```

```
GROUP BY Student.sno ;
```

➤ 当有元组新增操作时，DBMS会检查用户定义的完整性约束条件等，如不符合完整性约束条件，则将不会执行新增动作。



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组删除DELETE

➤ 元组删除DELETE命令: 删除满足指定条件的元组

DELETE FROM 表名 [**WHERE** 条件表达式];

➤ 如果WHERE条件省略, 则删除所有的元组。



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组删除DELETE(续)

➤ 元组删除DELETE示例

➤ 删除SC表中所有元组

```
DELETE FROM SC;
```

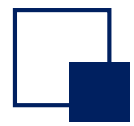
➤ 删除21030101号同学所选的所有课程

```
DELETE FROM SC WHERE sno = '21030101';
```

➤ 删除自动控制系的所有同学

```
DELETE FROM Student WHERE dno IN
```

```
(SELECT dno FROM Dept WHERE dname = '自动控制');
```



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组删除DELETE(续)

- 删除有四门不及格课程的所有同学

```
DELETE FROM Student WHERE sno IN  
(SELECT sno FROM SC WHERE score < 60  
GROUP BY sno HAVING COUNT(*) >= 4);
```

- 当有删除时，DBMS会检查用户定义的完整性约束条件等，如不符合完整性约束条件，则将不会执行删除动作。



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组更新UPDATE

- 元组更新UPDATE命令: 用指定要求的值更新指定表中满足指定条件 的元组的指定的列的值

UPDATE 表名

SET 列名 = 表达式 | (子查询)

[[, 列名 = 表达式 | (子查询)] ...]

[**WHERE** 条件表达式] ;

- 如果WHERE条件省略, 则更新所有的元组。
- 对多个列的更新操作分别用逗号 (,) 隔开。



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组更新UPDATE(续)

- 元组更新UPDATE示例
- 将所有教师工资上调5%

UPDATE **Teacher**

SET **salary** = **salary** * **1.05** ;

- 将所有计算机系的教师工资上调10%

UPDATE **Teacher**

SET **salary** = **salary** * **1.1**

WHERE **dno** **IN**

(**SELECT** **dno** **FROM** **Dept** **WHERE** **dname** = '计算机');



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组更新UPDATE(续)

- 当某同学001号课的成绩低于该课程平均成绩时，将该同学该门课成绩提高5%

UPDATE **SC**

SET **score** = **score** * 1.05

WHERE **cno** = '001' **AND** **score** < **SOME**

 (**SELECT** **AVG**(**score**) **FROM** **SC**

WHERE **cno** = '001');



3.4 SQL-DML之更新INSERT/UPDATE/DELETE

---- SQL-DML之元组更新UPDATE(续)

- 将张三同学001号课的成绩置为其班级该门课的平均成绩

UPDATE SC

SET score = (SELECT AVG(SC2.score)

FROM SC, Student S1, Student S2

WHERE S1.sclass = S2.sclass AND SC.sno = S2.sno

AND S1.sname='张三' AND SC.cno = '001')

WHERE cno = '001' AND sno IN (SELECT sno FROM Student WHERE

sname = '张三');



3.5 SQL-视图及DDL的进一步介绍

---- SQL-DDL的撤消与修改语句

- 基本表、数据库、视图等都可以撤消
- 撤消基本表

DROP TABLE 表名

- 例如，撤消学生表Student

DROP TABLE Student;

- 再例如，撤消教师表Teacher

DROP TABLE Teacher;



3.5 SQL-视图及DDL的进一步介绍

---- SQL-DDL的撤消与修改语句(续)

➤ 注意，SQL-DELETE语句只是删除表中的元组，而撤消基本表DROP TABLE的操作是撤消包含表格式、表中所有元组、由该表导出的视图、等相关的所有内容，所以使用要特别注意。

➤ 当表定义完成后，通常不是撤消表，而可能要修正表的定义，此时可使用ALTER TABLE语句

➤ 修正基本表的定义

ALTER **TABLE** tablename

[**ADD** {colname datatype, ...}]

增加新列

[**DROP** {完整性约束名}]

删除完整性约束

[**MODIFY** {colname datatype, ...}]

修改列定义



3.5 SQL-视图及DDL的进一步介绍

---- SQL-DDL的撤消与修改语句(续)

➤例如，在学生表Student(sno ,sname,ssex,sage, dno ,sclass)基础上增加二列saddr, pid

```
ALTER TABLE Student  
ADD saddr char[40], pid char[18];
```

➤再上例将表中sname列的数据类型增加两个字符

```
ALTER TABLE Student  
MODIFY sname char(10);
```

➤删除学生姓名必须取唯一值的约束

```
ALTER TABLE Student  
DROP UNIQUE ( sname );
```



3.5 SQL-视图及DDL的进一步介绍

---- SQL-DDL的撤消与修改语句(续)

➤ 撤消数据库

DROP DATABASE 数据库名;

➤ 例如撤消SCT数据库

DROP DATABASE SCT;



3.5 SQL-视图及DDL的进一步介绍

---- SQL-DDL的数据库指定与关闭命令

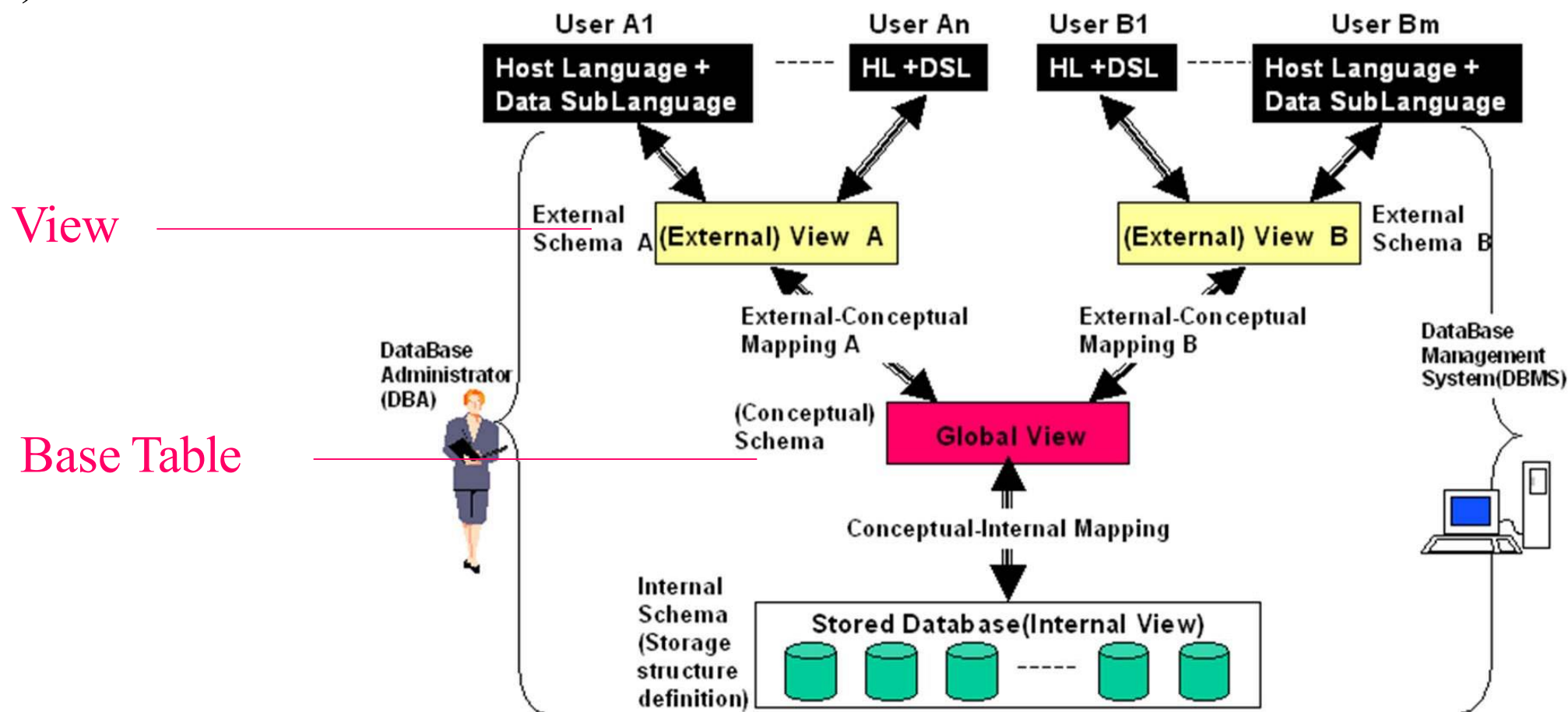
- 有些DBMS提供了操作多个数据库的能力，此时在进行数据库操作时需要 指定待操作数据库与关闭数据库的功能。
- 指定当前数据库
USE 数据库名;
- 关闭当前数据库
CLOSE 数据库名;



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的概念和结构

- 回顾我们学习过的三级模式两层映像结构
- 对应概念模式的数据在SQL中被称为基本表(Table), 而对应外模式的数据称为视图(View)

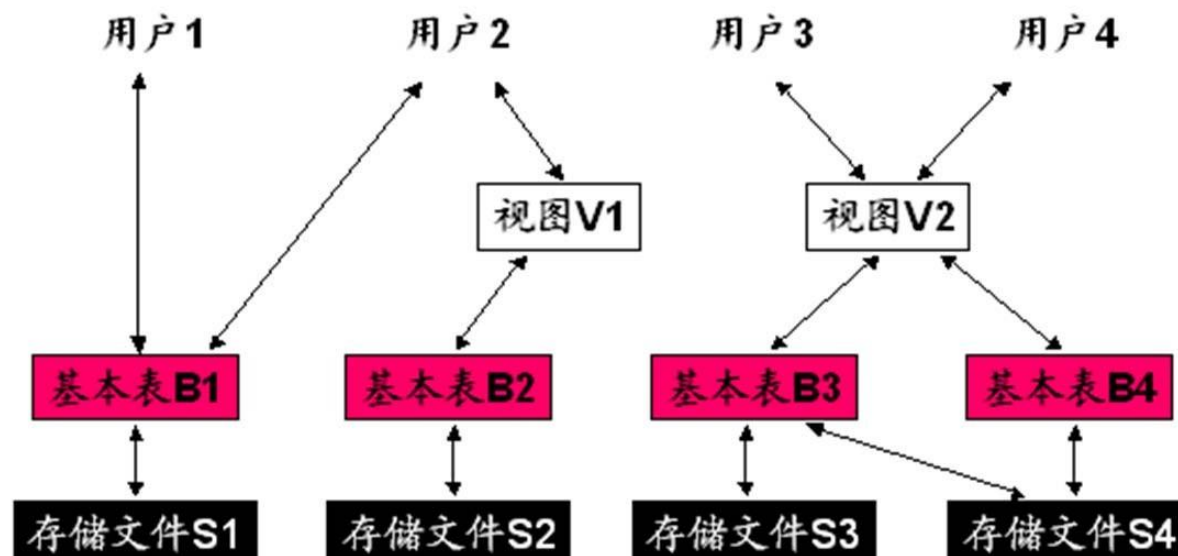


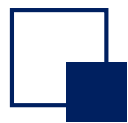


3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的概念和结构(View)

- SQL数据库结构
- 基本表是实际存储于存储文件中的表，基本表中的数据是需要存储的
- 视图在SQL中只存储其由基本表导出视图所需要的公式，即由基本表产生视图的映像信息，其数据并不存储，而是在运行过程中动态产生与维护的
- 对视图数据的更改最终要反映在对基本表的更改上。





3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的定义与使用

- 使用视图需要定义，定义视图采用CREATE VIEW语句
- 定义视图

```
CREATE VIEW view_name [(列名[ , 列名] ...)]  
AS 子查询 [with check option]
```

如果视图的属性名缺省，则默认为子查询结果中的属性名；也可以显式指明其所拥有的列名。

with check option指明当对视图进行INSERT，UPDATE，DELETE时，要检查进行INSERT/UPDATE/DELETE的元组是否满足视图定义中子查询中定义的条件表达式



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的定义与使用(续)

- 视图定义举例：定义一个视图 CompStud 为计算机系的学生，通过该视图可以将Student表中其他系的学生屏蔽掉。

以后如果对视图添加/更新/删除记录时，如果不是计算机的学生，就不让进行处理

```
CREATE VIEW CompStud AS
  ( SELECT * FROM Student
    WHERE dno IN ( SELECT dno FROM Dept
                  WHERE dname = '计算机' ) with check option ;
```

- 再例如定义一个视图Stud, 需要 sno, sname, sage 这些信息，之后只允许更新姓张的同学。

```
CREATE VIEW Stud AS
  ( SELECT sno , sname, sage FROM
    Student
    WHERE sname = '张%' with check option) ;
```



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的定义与使用(续)

- 再例如定义一个视图Teach为教师任课的情况（教师姓名、课程名字、学分），把Teacher表中的个人隐私方面的信息，如工资等屏蔽掉。

```
CREATE VIEW Teach AS  
    ( SELECT  T.tname, C.cname, credit  
      FROM    Teacher T, Course C  
      WHERE  T.tno = C.tno );
```



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的定义与使用(续)

➤ 当定义好视图后，视图可以像Table一样参与SQL的各种操作语句中使用。

➤ 例如，检索主讲数据库课程的教师姓名，我们可使用Teach

```
SELECT T.tname FROM Teach T  
WHERE T.cname = '数据库' ;
```

➤ 再例如，检索计算机系的所有学生，我们可使用CompStud

```
SELECT * FROM CompStud;
```

➤ 再例如，检索计算机系的年龄小于20的所有学生，我们可使用CompStud

```
SELECT * FROM CompStud  
WHERE sage < 20 ;
```



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的定义与使用(续)

- 定义视图，有时可方便用户进行检索操作。
- 例如，我们可定义一个视图StudStat, 描述学生的平均成绩、最高成绩、最低成绩等

```
CREATE VIEW StudStat(sno , sname, AvgS, MinS, MaxS, CNT)
AS ( SELECT sno , sname, AVG(score), MIN(score), Max(score), COUNT(*)
      FROM Student S, SC WHERE S.sno = SC.sno
      GROUP BY S.sno );
```

- 在定义了视图StudStat后，我们再检索某一学生平均成绩就很简单了：

```
SELECT sname, AvgS FROM StudStat WHERE sname = '张三' ;
```




互动小问题

➤ SQL视图更新操作是一个比较复杂的问题，因视图不保存数据，对视图的更新最终要反映到对基本表的更新上，而有时，视图定义的映射不是可逆的。例如：

```
CREATE VIEW S_G(sno ,Savg )  
AS ( SELECT sno ,AVG(score) FROM SC GROUP BY sno );
```

➤ 如要进行下述更新操作？

```
UPDATE S_G SET Savg = 85  
WHERE sno = '21030101';
```

能否由视图S_G的更新，而更新SC呢？



互动小问题

➤ 再例如：

```
CREATE VIEW ClassStud(sname, sclass)  
AS ( SELECT sname, sclass FROM Student );
```

如要进行下述更新操作？

```
INSERT INTO ClassStud VALUES ( '张三' , '2101101' );
```

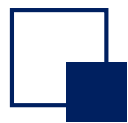
能否由视图ClassStud的更新，而更新Student呢？



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的更新

- SQL视图更新操作受到很大的约束，很多情况是不能进行视图更新的。
 - ❑ 如果视图的SELECT目标列包含聚集函数，则不能更新
 - ❑ 如果视图的SELECT子句使用了UNIQUE或DISTINCT，则不能更新
 - ❑ 如果视图中包括了GROUP BY子句，则不能更新
 - ❑ 如果视图中包括经算术表达式计算出来的列，则不能更新
 - ❑ 如果视图是由单个表的列构成，但并没有包括主键，则不能更新
- 对于由单一Table子集构成的视图，即如果视图是从单个基本表使用选择、投影操作导出的，并且包含了基本表的主键，则可以更新



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的更新

➤ 可更新SQL视图示例：

```
CREATE VIEW CStud(sno , sname, sclass)
AS (SELECT sno , sname, sclass FROM Student WHERE dno = '03');
```

➤ 上例是可以更新的

```
INSERT INTO Cstud
VALUES ( '210110101', '张三丰' , '2101101');
```

转换为



```
INSERT INTO Student
VALUES ( '210110101', '张三丰' , NULL, NULL, '03', '2101101');
```



3.5 SQL-视图及DDL的进一步介绍

---- SQL视图的撤消(续)

- 已经定义的视图也可以撤消
- 撤消视图

DROP VIEW view_name;

- 例如，撤消视图Teach

DROP VIEW Teach;

- 再例如，撤消视图CompStud

DROP VIEW CompStud;